

Chronicles_sprint3_v0

Realizzazione della Web GUI e rappresentazione delle entità osservabili

Obiettivi

Lo Sprint 3 rappresenta la fase conclusiva del progetto e si focalizza sulla realizzazione della **Web GUI**, come previsto dal requisito **GUI-1** definito nello Sprint 0 .

L'obiettivo principale è fornire una **rappresentazione dinamica e aggiornata dello stato della hold**, rendendo osservabili all'utente finale le informazioni prodotte dal sistema. In particolare, la GUI deve:

- Visualizzare lo stato degli **slot della stiva** (occupato/libero, PID associato)
- Aggiornarsi **in tempo reale** in risposta agli eventi del sistema (approccio reactive già introdotto nello Sprint 2)

Problem Analysis

Dall'analisi dei requisiti emerge che la GUI non è un semplice componente di visualizzazione statica, ma un **osservatore attivo del sistema distribuito**.

In particolare ci preoccuperemo di osservare e mostrare mediante interfaccia web:

- **Hold** → stato degli slot (occupazione, PID, peso)

Come evidenziato dall'analisi dei requisiti:

web-gui	Necessaria analisi del problema	Rappresenta dinamicamente lo stato della hold. Necessita Analisi Problema
---------	---------------------------------	---

1. Disaccoppiamento tra GUI e business logic

Seguendo i principi della **Clean Architecture**, la GUI non deve accedere direttamente allo stato interno degli attori, ma deve basarsi su:

- **Eventi pubblici**
- **Stream osservabili**
- **Risorse condivise aggiornate con CoAP**

2. Aggiornamento real-time

La GUI deve riflettere i cambiamenti senza polling inefficiente, si adotta quindi un modello **event-driven**, coerente con paradigma reactive.

La GUI deve inoltre tradurre la rappresentazione logica della hold in una **visualizzazione intuitiva** adatta ad un interfaccia web.

3. Trasporto delle informazioni

La GUI deve aggiornarsi in risposta agli eventi del sistema senza che il browser interroghi attivamente il server:

- Il polling HTTP introduce latenza artificiale e carico inutile su un sistema già event-driven.
- Il meccanismo CoAP observe notifica i client alla variazione della risorsa, ma il browser non parla CoAP nativo e necessita di un bridge.

WebSocket è la scelta naturale: stabilisce una connessione persistente full-duplex su cui il server pusha lo stato aggiornato nel momento esatto in cui **hold** chiama **updateSlot**, mantenendo la GUI allineata con il sistema senza overhead di polling e senza proxy aggiuntivi.

4. Interfaccia web

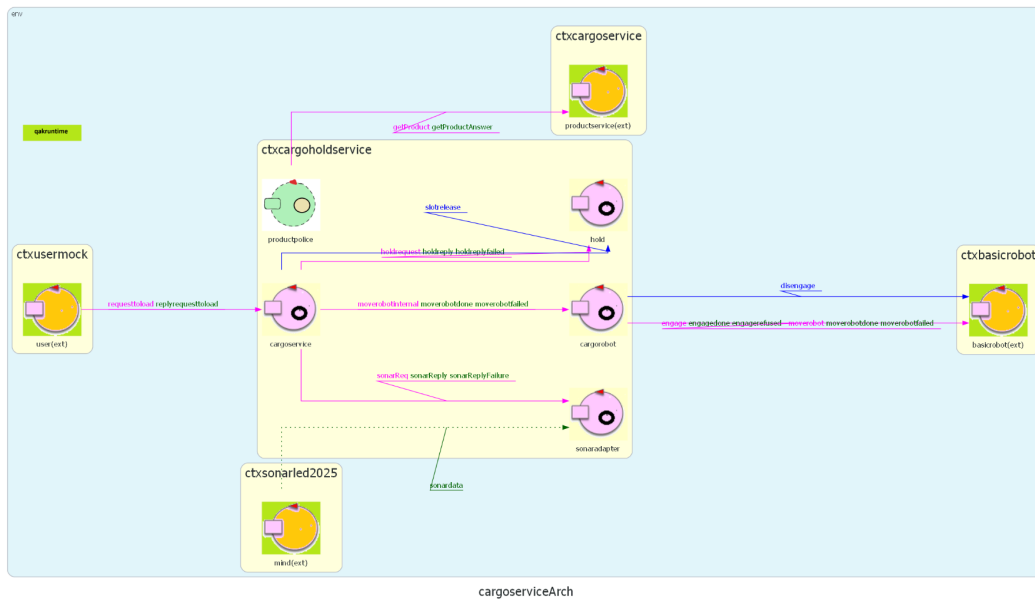
La visualizzazione delle informazioni è un problema risolvibile in mille modi diversi e tutti validi. Per gli scopi di questo progetto è sufficiente mostrare le informazioni, non essendo presenti vincoli o modelli “visivi” da rispettare.

Il modo più semplice per implementare una GUI web è una pagina HTML con JavaScript per aggiornare le parti dinamiche della pagina.

La GUI non è un componente monolitico ma un insieme di responsabilità distinte che, seguendo i principi della Clean Architecture, devono essere separate in layer indipendenti:

- **Risorsa condivisa** (**HoldStateResource**) — rappresenta lo stato osservabile della hold. È il confine tra il dominio e il layer di presentazione: nessun attore accede direttamente alla GUI, ma aggiorna questa risorsa che funge da contratto pubblico.
- **Server CoAP** (**CoapServerSetup**) — infrastruttura pura. Espone la risorsa sul protocollo CoAP e gestisce il meccanismo observe. Non conosce né la GUI né la logica di dominio.
- **Bridge WebSocket** (**HoldWebSocketServer**, **WsServer**) — adapter tra il mondo CoAP e il browser. Traduce le notifiche della risorsa in messaggi WebSocket senza conoscere il contenuto semantico dello stato.
- **HTML + script** (**hold.html**) — layer di presentazione puro. Riceve dati già pronti via WebSocket e li renderizza. Non contiene logica di dominio.
- **Observer di dominio** (attore **hold**) — l'unico componente che conosce lo stato degli slot e ha la responsabilità di aggiornare la risorsa condivisa. È il punto di contatto tra business logic e osservabilità: nessun altro attore tocca **HoldStateResource**.

Questa separazione garantisce che ogni componente possa evolvere indipendentemente, per esempio cambiare il formato JSON della risorsa o rimpiazzare il layer di presentazione non richiede modifiche al dominio.



PROJECT

Architettura della GUI

La Web GUI è stata progettata come un **client web disaccoppiato**, che mostra lo stato di una risorsa condivisa CoAP che viene aggiornata da parte di un attore.

La GUI si comporta come un **observer distribuito** che si sottoscrive ai topic di interesse:

- **holdstate** → stato slot

Dall'analisi del problema definiamo il sistema a supporto della GUI:

HoldStateResource.kt	Risorsa CoAP observable — singleton, aggiornato da hold, chiama <code>WsServer.broadcast()</code> ad ogni <code>updateSlot</code>
CoapServerSetup.kt	Avvio server CoAP Californium su porta 5683
GuiWebServer.kt	Server HTTP JDK su porta 8095 — serve esclusivamente <code>hold.html</code>
main/resources/static/hold.html	GUI web — connessione WebSocket su porta 8096, rendering dinamico 4 slot, event log, aggiornamento push senza polling
cargoservice.qak (modifica)	Blocco [# #] in cargoservice per avvio server; import e chiamate in hold

HoldWebSocketServer.kt	Classe che estende <code>WebSocketServer</code> — gestisce connessioni, invia snapshot iniziale in <code>onOpen</code>
WsServer.kt	Singleton — gestisce il ciclo di vita di <code>HoldWebSocketServer</code> ed espone <code>broadcast()</code> a <code>HoldStateResource</code>

HoldStateResource

Oggetto singleton Kotlin che estende `CoapResource` di Californium. Mantiene un array di 4 slot e risponde alle GET con payload JSON. La chiamata a `changed()` triggera la notifica CoAP observe a tutti i client registrati (espandibilità futura con collegamento a sistemi COaP centralizzati) e un broadcast su websocket per l'aggiornamento della GUI.

```
object HoldStateResource : CoapResource("holdstate") {
    init { isObservable = true }
    fun updateSlot(idx: Int, occupied: Boolean, pid: Int = 0, weight: Int = 0)
    {
        slots[idx] = Slot(idx, occupied, pid, weight)
        changed()
        WsServer.broadcast(toJson())
    }
}
```

Integrazione nel ciclo di vita QAK

Il main generato dal DSL viene rigenerato ad ogni salvataggio del file `.qak` — non è modificabile in modo persistente. La soluzione adottata sfrutta il blocco `[# #]` a livello di attore nel file `.qak`, che il DSL preserva ad ogni rigenerazione:

```
QActor cargoservice context ctxcargoholdservice {
    [#
        val _gui = run {
            it.unibo.ctxcargoholdservice.CoapServerSetup.start()
            it.unibo.ctxcargoholdservice.GuiWebServer.start()
            it.unibo.ctxcargoholdservice.WsServer.start()
        }
    #]
    ...
}
```

Il blocco `val _gui = run { ... }` è una property di inizializzazione della classe generata — viene eseguito una volta sola quando l'attore `cargoservice` viene istanziato dal framework, prima di qualsiasi stato. I fully qualified name eliminano la necessità di import.

Aggiornamento della risorsa da hold

L'attore `hold` è l'unico owner dello stato degli slot. Richiama `HoldStateResource` negli stati `reserve` e `handleRelease`, importando la risorsa con un singolo import nel blocco `[# #]` a livello attore:

```
QActor hold context ctxcargoholdservice {
    [#
```

```

import it.unibo.ctxcargoholdservice.HoldStateResource
...
#]
State reserve {
  [# HoldStateResource.updateSlot(lastIdx, true, lastPid, lastPeso) #]
  ...
}
State handleRelease {
  [# HoldStateResource.updateSlot(idx, false) #]
  ...
}
}

```

Configurazione delle porte

HoldStateResource (CoAP)	5683	UDP
GuiWebServer (HTTP)	8095	HTTP
WsServer	8096	WebSocket

Conclusion

Lo Sprint 3 completa il sistema introducendo un livello fondamentale: **l'osservabilità**.

La Web GUI non è solo un'interfaccia grafica, ma un componente architetturale che:

- espone lo stato interno del sistema
- rispetta i principi della reactive programming
- mantiene il disaccoppiamento tra livelli

Il risultato finale è un sistema distribuito, modulare e monitorabile in tempo reale, in linea con i requisiti iniziali e con le scelte architetturali adottate nei precedenti sprint.